

prfaq

A virtual product manager for engineers building with AI coding tools

February 2026 | Jim Freeman | punt-labs

Press Release

Summary

SAN FRANCISCO — Today, punt-labs released `prfaq`, a free, open-source Claude Code plugin that guides engineers and solo founders through Amazon’s Working Backwards PR/FAQ process to evaluate product ideas before writing code. Unlike blog posts, templates, or workshops that sit outside the developer’s workflow, `prfaq` runs as a slash command inside the same Claude Code session where the engineer already builds software, producing a professionally typeset PDF in under an hour. Engineers who ship features at unprecedented speed can now apply the same rigor to deciding *which* features to ship.

Problem

AI coding tools have made it possible for a single engineer to build in a weekend what used to take a team a quarter. The bottleneck has shifted: the hard part is no longer writing code — it is knowing what to build and why. Engineers and solo founders using Claude Code, Cursor, and similar tools ship fast, but most lack formal product training. They skip customer definition, skip competitive analysis, skip unit economics — and discover after burning through tokens and weeks of building that the product does not resonate.

The consequences are measurable. Industry analysts estimate that \$1.5 trillion in technical debt will accumulate by 2027 from AI-generated code, much of it from products that solve the wrong problem or serve an undefined customer. Roughly 8,000 startups built during the current vibe coding wave will require significant rebuilds. The failure mode is not bad code — it is code that solves the wrong problem with confidence.

Today, an engineer who wants product discipline has three options: hire a product manager they cannot afford, read a book and try to apply frameworks manually, or just iterate — ship something, get feedback, ship again. Iteration works. Some of the best developer tools started as something an engineer built for themselves, and others appreciated. But iteration has real costs: every cycle burns tokens, burns calendar time marketing to early users, and burns the founder’s energy. Getting customer feedback is often the actual bottleneck. Plenty of engineers would develop stronger product sense if the frameworks were more accessible — available in the workflow where they already build, not in a book or a workshop they will never attend.

Solution

`prfaq` brings product thinking directly into the engineer’s development environment. By typing `/prfaq` in any Claude Code session, the engineer starts a structured conversation: Claude

asks about the target customer, the problem, the competitive landscape, and the business model. It asks hard questions — the kind a product manager would ask before greenlighting a project.

From these answers, Claude generates a complete PR/FAQ document: a one-page mock press release describing the product as if it has already shipped, followed by external FAQs (what a customer would ask) and internal FAQs (what leadership, finance, and engineering would ask). The document is evaluated against Marty Cagan's four risks framework — value, usability, feasibility, and viability — producing an honest assessment of whether the product is worth building.

The output is a LaTeX-compiled PDF: a clean, professional artifact suitable for sharing with co-founders, investors, or advisors. It is not a disposable brainstorm. It is a decision-making document designed to be read, debated, and revised.

Customer Quote

*"I kept building MVPs that nobody used. I'd spend three weeks coding, show it to people, and get polite shrugs. **prfaq** made me realize I was solving my own problem, not my customer's. The PR/FAQ took forty-five minutes and saved me a month of wasted work. The hardest question it asked was 'what evidence do you have that customers want this?' I didn't have any. That was the answer."*

— **Marcus Okonkwo**, Solo Founder, SyncPilot

Getting Started

Getting started with **prfaq** takes three steps. First, run the one-line installer: `curl -fsSL https://punt-labs.github.io/prfaq/install | bash`. This clones the plugin repository and registers it with Claude Code automatically. Second, open any Claude Code session and type `/prfaq`. Third, answer the discovery questions — Claude guides the conversation, asks follow-ups, and drafts the PR/FAQ document. Within an hour, the engineer has a compiled PDF that forces clarity on customer, problem, differentiation, and business model. No account required, no subscription, no configuration.

Spokesperson Quote

*"Engineers are the best product thinkers in the room — they just don't always have the frameworks. We built **prfaq** because the engineers using Claude Code every day are also the ones making product decisions, and they deserve a tool that meets them where they already work. The PR/FAQ process has been proven at Amazon for two decades. We didn't invent it. We made it accessible inside a terminal."*

— **Jim Freeman**, punt-labs

Call to Action

prfaq is free and open source, available now at <https://github.com/punt-labs/prfaq>. Install it, type `/prfaq`, and find out if your next product idea is worth building before you build it.

Frequently Asked Questions

External FAQs

Q: What is prfaq and who is it for?

prfaq is a free Claude Code plugin that guides engineers and solo founders through the Amazon Working Backwards PR/FAQ process. It is for anyone who builds software with AI coding tools and wants to validate a product idea before committing weeks or months of development time. If you make product decisions — what to build, for whom, and why — this tool is for you.

Q: How is this different from using a PR/FAQ template?

A template is a blank page with headings. prfaq is an interactive process: Claude asks discovery questions, challenges weak answers, identifies gaps in customer evidence, and generates the document from your responses. It applies the four risks framework to evaluate your idea honestly, not just format it. A template tells you *what* to write. prfaq helps you figure out *what to think*.

Q: How do I get started?

Run the one-line installer (`curl -fsSL https://punt-labs.github.io/prfaq/install | bash`), then type `/prfaq` in any Claude Code session. The plugin walks you through the entire process. No account, no API keys, no configuration. Installation takes under a minute.

Q: What does the output look like?

A professionally typeset PDF compiled from LaTeX. The document includes a one-page press release, external and internal FAQs, and a four-risks assessment. It is designed to be shared with co-founders, investors, or advisors — not a throwaway brainstorm document.

Q: Do I need to know LaTeX?

No. The plugin generates the LaTeX source and compiles it automatically. You never see or edit LaTeX unless you choose to. You need a TeX distribution installed (TeX Live or similar), which the installer checks for and provides setup instructions if missing.

Q: Can I iterate on the document?

Yes. The plugin generates a `.tex` file in your project directory. You can re-run the process, edit sections, or ask Claude to revise specific parts. The Working Backwards process is designed for rapid iteration — the point is to iterate on a one-page document instead of iterating on a shipped product.

Internal FAQs

Value & Market

Q: What is the total addressable market?

The primary market is engineers and solo founders using AI coding tools to build products. As of early 2026, over 90% of US-based developers use AI coding assistants daily. Claude Code specifically has reached an estimated \$1–2B in annual recurring revenue and is the fastest-growing AI coding tool by several independent surveys. The subset of these users who are making product decisions — solo founders, indie hackers, startup engineers, and technical co-founders — numbers in the hundreds of thousands. This is not a TAM-based revenue argument (the plugin is free); it is a reach argument. A useful, free plugin in a fast-growing ecosystem reaches users through organic discovery.

Q: What evidence do we have that customers want this?

Three categories of evidence. First, the “vibe coding” phenomenon: industry data shows thousands of products being built at speed without product discipline, with analysts tracking 8,000+ startups expected to need significant rebuilds due to lack of upfront product thinking. Second, the explosion of “how to think about product as an engineer” content — books, courses, and newsletters targeting technical founders suggest unmet demand for product skills in developer contexts. Third, the Working Backwards process itself has two decades of validation at Amazon, where it is mandatory for new product proposals. The gap is not whether the framework works — it is accessibility. Engineers do not attend PM workshops. They install plugins.

Q: Who are the competitors and why will we win?

Direct competitors are product management tools (Productboard, Aha!, Notion templates) and business planning tools (Lean Canvas generators, business model canvas apps). None of these are integrated into a developer’s coding environment. They all require context-switching: leave the terminal, open a browser, fill in a form. The structural advantage is integration. A Claude Code plugin runs in the same session where the engineer builds software. The mental model shift from “coding” to “product thinking” happens in a single command, not a tab switch. If competitors respond, they would need to build Claude Code plugins themselves — which validates the distribution channel.

Technical

Q: What are the major technical risks?

The technical risks are minimal. The plugin is a set of markdown files (skill definition and reference guides) plus a LaTeX template and a shell script. It requires no backend, no API, no database, no infrastructure. The primary technical dependency is the Claude Code plugin system, which is stable and well-documented. The LaTeX dependency requires users to have a TeX distribution installed, which is a mild friction point for users who do not already have one. Mitigation: the installer checks for TeX and provides platform-specific setup instructions.

Q: What dependencies exist on other teams or systems?

The plugin depends on Anthropic’s Claude Code plugin system for loading and execution, and on a local TeX distribution for PDF compilation. Both are external dependencies outside our control. The Claude Code plugin API is stable (v1), and TeX Live is a mature, 30-year-old ecosystem. If Anthropic changes the plugin format, the migration would be straightforward — the plugin is a handful of files with simple structure. If TeX Live is unavailable, the plugin still generates the .tex source file; only PDF compilation is affected.

Q: What is the estimated development timeline?

The plugin is built. Initial development took one session. Ongoing work is iterative: improving the skill prompts based on real usage, expanding reference material, and potentially adding alternative output formats. There is no multi-month roadmap because there is no infrastructure to build.

Business

Q: What is the revenue model?

There is none. prfaq is free and open source under a permissive license. The strategic value is threefold: (1) it is a proving ground for the Claude Code plugin development pattern, informing future commercial plugins; (2) it builds reputation and visibility in the Claude Code ecosystem; (3) it creates a feedback loop for understanding how engineers approach product thinking, which informs other products at punt-labs.

Q: What are the key metrics for success?

Three metrics, measured via GitHub analytics and community signals:

1. **GitHub stars and forks** — target: 500 stars within 6 months. This measures discoverability and perceived value.
2. **Issues and pull requests from external contributors** — target: 10 external PRs within 6 months. This measures whether the plugin is useful enough that people invest time improving it.
3. **Mentions in developer communities** (Hacker News, Twitter/X, Reddit, Discord servers) — target: 3 organic mentions within 3 months. This measures whether the plugin solves a real problem that people tell others about.

If none of these targets are met after 6 months, the plugin is not reaching its audience and the distribution strategy should be reconsidered.

Q: Why now? What has changed?

Three shifts converged. First, AI coding tools reached mainstream adoption in 2025–2026, creating a large population of engineers who can build products independently but lack product discipline. Second, Claude Code’s plugin system matured, making it possible to deliver structured workflows inside the developer’s existing tool. Third, the consequences of “vibe coding without product thinking” are becoming visible — failed launches, rebuilds, wasted months — creating demand for lightweight product frameworks that do not require hiring a PM or leaving the terminal.

Q: What are we not building?

We are not building a SaaS product management platform. We are not building a competitor to Jira, Linear, Productboard, or Notion. We are not adding collaboration features, dashboards, or analytics. We are not building a web interface. The plugin is a single-player tool that runs locally, produces a document, and gets out of the way. Scope discipline is the feature.

Risk Assessment

Four Risks Assessment

Value — Medium

Engineers and solo founders demonstrably lack product discipline, and the consequences (wasted builds, failed launches) are real and growing. However, the evidence is market-level, not customer-level — we have not yet interviewed individual users who say “I would use a PR/FAQ plugin in my terminal.” The demand for the *framework* is proven (Amazon, two decades). The demand for *this delivery mechanism* is hypothesized. Risk mitigation: the plugin is free, so the adoption barrier is low — users risk minutes, not money.

Usability — Low

One-line install, one slash command to start. Claude guides the entire process interactively. No configuration, no account creation. The LaTeX dependency is the main friction point; the installer checks for it. Time to value is under one hour.

Feasibility — Low

The plugin is built and functional. No backend, no infrastructure, no ongoing operational cost. The hard engineering problem (interactive AI guidance) is solved by Claude itself. The plugin is structured prompts and a document template.

Viability — Low

No revenue target, no operational cost, no support burden. Strategic asset for learning plugin development patterns and building ecosystem presence. The only viability risk is opportunity cost, which is minimal given the small codebase (under 20 files, no dependencies beyond TeX).